Architecture and Engineering Decisions Report

Project: Lunar Telemetry Data Pipeline: Real-Time Ingestion with Kafka & InfluxDB

1. System Architecture & Network Segregation

The primary objective of this architecture is to simulate a realistic, fault-tolerant telemetry pipeline between a Lunar Surface Rover and a Gateway Orbiter. To enforce strict network isolation (simulating an air-gapped environment), the system is divided into two distinct Docker networks:

- **deep-space-net:** Hosts the Python telemetry producers and the Apache Kafka broker.
- **ground-control-net:** Hosts the InfluxDB time-series database and the Mission Dashboard.

Architectural Decision - Telegraf as a Bridge: Telegraf is configured as the sole ingestion agent bridging both networks. It consumes from Kafka and writes to InfluxDB, ensuring that the database is completely isolated from the raw data producers. **Architectural Decision - Kafka Port Exposure:** The port 9092 for Kafka was intentionally removed from the exposed ports in the compose.yaml. Since all communication between the producers, Kafka, and Telegraf occurs internally within the Docker network, exposing this port to the host machine was deemed unnecessary and a potential security flaw.

2. Mission Dashboard Design

The Mission Control dashboard was designed to provide both real-time operational awareness and historical context. In addition to the four standard fixed graphs, specific visualizations were implemented to monitor critical subsystem behaviors:

- **Scatter Plot (SNR vs Latency - COMMS):** This visualization correlates Signal-to-Noise Ratio with Latency. In a healthy link, high SNR corresponds to low latency. Degraded communications appear as a distinct, separated cluster of points on the scatter plot.
- **Histogram (Latency Distribution):** Designed to clearly distinguish between normal operation modes (~20-30 ms) and degraded modes (200-800+ ms).
- **Latest Telemetry Snapshot (Table):** A dynamic table providing a single-row summary per subsystem with the most recent values, ensuring Ground Control has an immediate overview of the rover's status.

3. Flux Query Engineering

To build the dashboard, advanced Flux scripting was required to manipulate the time-series data efficiently. Key implementations include:

- **Data Reshaping (pivot):** InfluxDB natively stores data in a vertical format (one row for RPM, another for traction, etc.). The pivot() function was used with rowKey: ["_time", "host", "unit"] and columnKey: ["_field"] to transform the data into a wide format, creating a single row per timestamp with distinct columns for each metric, avoiding collisions between series.
- **Dynamic Threshold Lines (map & union):** To visualize critical limits (e.g., critical battery discharge), the map() function was used to create a static value across the time range. The union() function then combined the actual telemetry data with this mapped threshold, allowing both the real curve and the red horizontal threshold line to be rendered on the same graph.
- **Aggregation (aggregateWindow & max):** To prevent performance degradation when zooming out, aggregateWindow was used to calculate means over specific periods (e.g., 10s, 30s), while setting createEmpty: false to handle the burst-nature of the data. For critical safety metrics, max() was used to isolate the peak values within the visible time range.

4. Alerting Strategy

Ground Control requires an automated alerting system to detect anomalies. The alerting architecture was evaluated against two approaches: external webhooks (e.g., Slack/HTTP) and native InfluxDB Checks.

Decision - Native InfluxDB Checks: While configuring a local HTTP endpoint or Slack webhook is standard in enterprise environments, native InfluxDB Checks were selected as the optimal solution for this project. This ensures the pipeline remains entirely self-contained and reproducible. Anyone running the docker-compose can trigger and view the alerts natively in the InfluxDB UI without depending on external API configurations or internet connectivity.

Three primary Flux alert checks were implemented directly in InfluxDB:

1. **Alert ECLSS (Radiation):** Monitors radiation spikes beyond safe commercial flight limits.
2. **Alert HGA (SNR):** Monitors signal degradation drops.
3. **Alert Mobility (Battery):** Monitors battery discharge. *Note:* While battery voltage was the raw metric, it was logically treated and represented as a Battery Percentage (%) across both the producers and the dashboard.

Future Enhancements: Due to time constraints during the initial sprint, an alert for Cabin Pressure anomalies was omitted. Implementing a pressure drop alert would be the immediate next step for the ECLSS monitoring module.

5. Team Collaboration

The architecture, infrastructure deployment, and Flux scripting for this project were successfully developed and integrated through a collaborative team effort. Tasks were divided between data simulation (producers), infrastructure orchestration (Docker), and database analytics (Flux/Dashboards), ensuring a cohesive and resilient final pipeline.